

lux Connection-Lease Reaping — Fidelity Control (Self-Reap on Write)

Buggy variant: Claim sweeps its own caller, never renewing it first

September 2026

1 Introduction

This is the fidelity control for H3 in `docs/connection_lease_reaping.tex`, not the design. It removes only the self-protection `Claim` otherwise gives its own caller: no renewal before the sweep, and no exclusion of the caller from the set the sweep releases. `HubDisplay.apply` calls `self._reap_and_release()` – an unconditional sweep of *every* lapsed session under `HubClientRegistry.reap` – before it does anything else, and never calls anything that would renew the calling connection’s own lease first. A connection that only ever contacts the Hub through writes (an initial `show`, then only patches) never renews outside that first call; once its lease’s length has passed, its own next write sweeps and releases itself before performing the very write that should have kept it alive. Read the main specification first; this document states only what differs, in Section 6.1.

2 Basic Types

$[CONN, IDENT, SCENE]$

Unchanged from the main specification.

3 Free Types

$TSTATE ::= tup \mid tdown$

$LSTATE ::= llive \mid llapsed$

$OWNER ::= unowned \mid conn\langle\langle CONN \rangle\rangle$

Unchanged from the main specification.

4 State

ConnReg

$registered : \mathbb{P} CONN$
 $identity : CONN \leftrightarrow IDENT$
 $transport : CONN \leftrightarrow TSTATE$
 $lease : CONN \leftrightarrow LSTATE$
 $owner : SCENE \rightarrow OWNER$

$registered \subseteq \text{dom } identity$
 $\text{dom } identity = \text{dom } transport$
 $\text{dom } transport = \text{dom } lease$

Unchanged from the main specification.

5 Initialization

<i>Init</i>	
<i>ConnReg'</i>	
<i>registered'</i>	$\emptyset$
<i>identity'</i>	$\emptyset$
<i>transport'</i>	$\emptyset$
<i>lease'</i>	$\emptyset$
<i>owner'</i>	$(\lambda s : SCENE \bullet unowned)$

Unchanged from the main specification.

6 Operations

6.1 Claim — BUGGY: no self-renewal, no self-exclusion from the sweep

This is the one operation that differs from the design. The ownership-gated write itself is unchanged – the claim guard and the grant are exactly as in the fixed spec. What is missing is the special treatment the fixed **Claim** gives its own caller: *lease'* is left unchanged instead of renewing *c?* to *llive* first, and the sweep set drops the $c2 \neq c?$ exclusion, so it reads *every* registered connection whose lease has lapsed – *c?* included, exactly as `HubClientRegistry._sweep_locked` sweeps every session in `_sessions`, caller or not, when `HubDisplay._reap_and_release` invokes it unconditionally at the top of `apply`.

<i>Claim</i>	
$\Delta ConnReg$	
<i>c?</i> : <i>CONN</i>	
<i>s?</i> : <i>SCENE</i>	
<i>c?</i> $\in$ <i>registered</i>	
<i>transport c?</i> = <i>tup</i>	
<i>lease'</i> = <i>lease</i>	
<i>registered'</i> = <i>registered</i> \	
$\{c2 : CONN \mid c2 \in registered \wedge lease\ c2 = llapsed\}$	
$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$	
$c2 \in registered \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$	
$\times \{unowned\}))\ s? = unowned$	
$\vee$	
$(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$	
$c2 \in registered \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$	
$\times \{unowned\}))\ s? = conn(c?)$	
<i>owner'</i> = $(owner \oplus (\{s : SCENE \mid \exists c2 : CONN \bullet$	
$c2 \in registered \wedge lease\ c2 = llapsed \wedge owner\ s = conn(c2)\}$	
$\times \{unowned\})) \oplus \{s? \mapsto conn(c?)\}$	
<i>identity'</i> = <i>identity</i>	
<i>transport'</i> = <i>transport</i>	

6.2 Connect, Renew, TransportDies, LeaseLapse, Read, Depart, TimedReap — unchanged

Identical to the main specification: only **Claim**'s self-protection is removed.

Connect

$\Delta ConnReg$
 $c? : CONN$
 $i? : IDENT$

$registered' = registered \cup \{c?\}$
 $identity' = identity \oplus \{c? \mapsto i?\}$
 $transport' = transport \oplus \{c? \mapsto tup\}$
 $lease' = lease \oplus \{c? \mapsto llive\}$
 $owner' = owner$

Renew

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $lease' = lease \oplus \{c? \mapsto llive\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$

TransportDies

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $transport' = transport \oplus \{c? \mapsto tdown\}$
 $registered' = registered$
 $identity' = identity$
 $lease' = lease$
 $owner' = owner$

LeaseLapse

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llive$
 $lease' = lease \oplus \{c? \mapsto llapsed\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$

Read

$\Xi ConnReg$

Depart

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $registered' = registered \setminus \{c?\}$
 $owner' = owner \oplus$
 $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$

TimedReap

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = lapsed$
 $registered' = registered \setminus \{c?\}$
 $owner' = owner \oplus$
 $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$

7 Verification: the goal that must flip

Against this variant, I5/I6's negation must be *found* — the trace in `connection_lease_reaping.tex`, Section H3, reaches it in four steps and never calls `TransportDies` at all: `Connect(c1, i1)`; `Claim(c1, s1)`; `LeaseLapse(c1)`; `Claim(c1, s1)` again.

```
# must be FOUND against this variant (NOT found against the fixed spec)
probccli docs/connection_lease_reaping_self_reap_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
      not(c : registered) & transport(c) = tup))" \
-p DEFAULT_SETSIZE 3
```

The same trace also satisfies the more general I2/I5 goal, since a transport that never died is one instance of a connection that is absent from `registered` while still owning a scene:

```
# must be FOUND against this variant (NOT found against the fixed spec)
probccli docs/connection_lease_reaping_self_reap_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
      not(c : registered)))" \
-p DEFAULT_SETSIZE 3
```

I1 must stay unaffected — this variant does not touch `TransportDies` or `Depart`, so the chain `TransportDies`; `Depart` still completes exactly as in the fixed spec:

```
# must remain FOUND
probccli docs/connection_lease_reaping_self_reap_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
      transport(c) = tdown)" \
-p DEFAULT_SETSIZE 3
```